

PyQCU 全库代码分析（最终版 v3）

多线程多卡 MultiGrid：竞争修复、2.8x 构建加速与大格子验证

opencode analy

2026-08-14

摘要

本报告为 PyQCU 全库代码分析的最终版（v2），聚焦本会话（8 小时高强度 debug/optim/新功能循环）的成果。PyQCU 是 Lattice QCD 的 Python/Cython 库：CUDA 加速的 Wilson/Clover Dirac 算子、BiStabCG 与多重网格（MultiGrid）求解器、stout smearing 与规范场生成，MPI 分布于 4D 进程网格。本版核心内容：多线程多卡（一线程一卡）MultiGrid 路径的完整构建——三处 CudaSchurOp bug 修复、C++ 私有流与 torch 分配器深层竞争修复、多线程 null 向量与 33-tensor stencil 构建（加速 2.8x）、独立问题吞吐模式、Python multigrid CUDA 混合路径同步与 BiCGStab 自动重启。全部结论附实测证据与参考源。

目录

1 仓库概览（截至 2026-08-14 15:00）	1
2 多线程多卡（一线程一卡）MultiGrid 路径	1
2.1 架构三件套	2
2.2 本轮关键 bug 修复（证据驱动）	2
2.3 多线程粗网格构建（性能优化）	2
2.4 性能与正确性实测（RTX 4060 单卡）	3
3 对象映射（代码符号 物理对象）	3
4 关键代码片段	3
4.1 CudaSchurOp 深层竞争修复	3
4.2 多线程 null 向量生成（鲁棒化）	4
5 参考源清单	5
6 大格子与多配置验证（v3 新增）	5
7 结论与局限	6

1 仓库概览（截至 2026-08-14 15:00）

类别	数量	说明
Python 文件	122	库 + 测试 + Cython 源
C++ 头 / CUDA 源	24 / 30	cpp/cuda/qcu/ 模板内核
文档 (md)	134	AGENTS.md 体系 + docs/
git 提交 (主分支)	1085+	本会话新增 7 个 [opencode] 提交

架构层级:入口层(pyqcu/、examples/)→核心层(dslash/solver/smear/tools)→桥接层(pyqcu/cuda/Cython 桥)→后端层 (cpp/cuda/qcu/)→兼容层 (cann 占位)。参数协议: params(int32[54]) / argv(float[7]) / set_ptrs(int64[100]) 三张扁平张量桥接(pyqcu/cuda/define.py:11-90),与 define.h 同步; 调用生命周期要求调用间 _SET_INDEX_ 递增。

2 多线程多卡（一线程一卡）MultiGrid 路径

2.1 架构三件套

- 1. **Cython 桥真并行**: pyqcu/cuda/qcu/qcu.pyx 全部桥函数 GIL 段取指针、with nogil 调 C++; pxd 声明带 nogil 关键字、别名 cimport (qcu_api.pxd) 规避同名 def 覆盖。
- 2. **多线程安全 Schur 算子**: CudaSchurOp (pyqcu/cuda/_schur_op.py:47) 每实例独立 params/set_ptrs 副本 + 线程安全槽位分配器。
- 3. **多线程多卡驱动**: MultiGpuMultigrid (pyqcu/cuda/_multi_gpu.py:133) 单进程 N 线程 × 卡绑定; 一致性模式 (共享输入) + 独立问题模式 (independent_problems, 每线程不同 seed、缓存按线程分目录)。

2.2 本轮关键 bug 修复（证据驱动）

- 1. **CudaSchurOp 三处 bug**: 构造克隆模块级 params (默认 32³ 格点) 与 gauge/clover 维度不符 → 内核越界 (illegal memory access); 输出张量非连续时 C++ 写入 contiguous 副本、原张量不变 (垃圾); 默认流与 C++ 私有流无同步。修复后 100 次 matvec 与 Python 参考一致 (2 × 10⁻⁷)。
- 2. **深层竞争**: C++ 私有流写 torch 分配器内存(池复用)在”预计算参考 + 大量中间张量”场景下 25-80% 调用结果错误 (‘precomputed-refs’ 复现: 8x8x8x16 probe 输入)。修复: CudaSchurOp.matvec 输出改**预分配固定缓冲** + 设备级同步 + clone (_schur_op.py:75-95)。修复后 300/300 通过。
- 3. **Python multigrid CUDA 混合路径**: cycle 的 _matvec 等 4 处 C++ 调用后补 torch.cuda.synchronize() (固定输出缓冲 + 私有流竞争 → iter 0 breakdown); BiCGStab breakdown 改**自动重启** (保留 x/r、重置影子向量与系数、阈值 1e-20 + alpha/omega 有限性检查)。实测 8x8x8x8 1L 残差 8.7 × 10⁻⁷、8x8x8x16 2L 7.8 × 10⁻⁷。

2.3 多线程粗网格构建（性能优化）

- `give_null_vecs_mt` (`tools/_multigrid.py:281`): 每线程独立 CUDA RNG generator (防全局 RNG 竞争产生相关序列)、`nan/breakdown` 重试 + 随机回退、worker 内显式 `set_device`、`nthreads<=1` 直接执行 (防嵌套线程池)。
- `build_stencil_mt` (`tools/_multigrid.py:365`): `probe` 探测点写集不相交; `src_c` 预分配复用 (省 1536 次分配)。
- `build_schur_levels` 多线程分支 (`_multi_gpu.py:45`): C++ matvec 构建 (替代 Python einsum) $\rightarrow 8 \times 8 \times 8 \times 16$ E=24 构建 **135s $\rightarrow$ 48s (2.8x)**, stencil 自洽性 2.0×10^{-7} ; C++ MG 用多线程构建的粗算子残差 1.6×10^{-7} 。

2.4 性能与正确性实测 (RTX 4060 单卡)

验证项	配置	结果	判据
多线程一致性	2/4 线程 $\times 8 \times 8 \times 8 \times 16$	线程间 $< 10^{-5}$ PASS	tol 1e-5
C++ MG 残差	mt 构建粗算子 2L	1.6×10^{-7} PASS	$< 10^{-5}$
构建加速	$8 \times 8 \times 8 \times 16$ E=24	135s $\rightarrow$ 48s (2.8x)	实测
独立问题模式	2 线程不同 seed	解不同、各自收敛 PASS	解差 $> 10^{-4}$
参数扫描	8 配置 (L2/3、r5-20、ct1e2-1e5)	全部 1.5×10^{-7} 收敛	sweep
Python multigrid	1L/2L 混合路径	8.7×10^{-7} / 7.8×10^{-7}	$< 10^{-5}$
h5py 并发	4 线程独立文件	max_err=0 PASS	往返相等
test12 clean	$8 \times 8 \times 8 \times 16$ 2L	MG 残差 8.3×10^{-7} PASS	权威回归

3 对象映射（代码符号 物理对象）

代码符号	对象概念	物理含义
<code>hopping / sitting</code>	Wilson 跃迁 / 质量项	$\bar{\psi} \gamma_{\mu} U_{\mu} \psi$ 与对角项
<code>clover_term</code>	Clover 项	$c_{sw} \sigma_{\mu\nu} F_{\mu\nu}$, $O(a)$ 改进
<code>matvec_parity</code>	Schur 奇偶算子	$S = A_{oo} - k^2 D_{oe} A_{ee}^{-1} D_{eo}$
<code>null_vecs</code>	近零空间向量	逆迭代 $v - A^{-1} A v$ (残差级噪声向量)
<code>33-tensor stencil</code>	粗网格算子	Galerkin $A_c = P^T S P$ (on-site+ 近邻 + 对角)
<code>CudaSchurOp</code>	C++ Schur 算子封装	多线程安全, 每实例独立状态
<code>MultiGpuMultigrid</code>	多卡 MG 驱动	一线程一卡, 一致/独立两种模式

4 关键代码片段

4.1 CudaSchurOp 深层竞争修复

```

if params is not None:
    self.params = params.clone()
else:
    self.params = _module_params.clone()
self.params[define._SET_INDEX_] = _GLOBAL_SET_ALLOC.next()
self.set_index = int(self.params[define._SET_INDEX_])
self.params[define._SET_PLAN_] = 1
self.params[define._VERBOSE_] = 0
# 独立 set_ptrs 副本：多线程各持一份，互不干扰
self.set_ptrs = _module_set_ptrs.clone()
self._g, self._ce, self._coo, self._cei, self._coi = g, ce, coo, cei, coi
self._y_buf = None
self._y_buf_shape = None
qcu.applyInitQcu(self.set_ptrs, self.params, av)

def matvec(self, x_o):
    # 输入/输出必须连续：桥函数 .contiguous() 会拷贝，若输出非连续则
    # C++ 写入副本、原张量内容不变（垃圾）导致求解错误。
    x_c = x_o.contiguous()
    # C++ 输出写预分配固定缓冲（不经过 torch 分配器，避免池复用与
    # C++ 私有流之间的深层竞争——实测 torch 池复用时偶发结果错误）；
    # 设备级同步保证 C++ 私有流完成，再 clone 到新张量（默认流语义）。
    if self._y_buf is None or self._y_buf_shape != tuple(x_c.shape):
        self._y_buf = torch.empty_like(x_c)
        self._y_buf_shape = tuple(x_c.shape)
    torch.cuda.current_stream().synchronize()
    qcu.applyCloverBistabCgDslashQcu(
        self._y_buf, x_c, self._g, self._ce, self._coo, self._cei, self._coi,
        self.set_ptrs, self.params)
    torch.cuda.synchronize()
    return self._y_buf.clone()

def release(self):

```

4.2 多线程 null 向量生成（鲁棒化）

```

def _gen(op, i, tid):
    # 每向量独立 CUDA generator（线程内顺序使用，无竞争）
    gen = torch.Generator(device=device).manual_seed(
        (seed + tid * 104729 + i * 10007) % (2**31 - 1))
    mv = op.matvec if hasattr(op, 'matvec') else op
    v = None
    for _try in range(5):
        v = torch.randn([e] + list(lat_fine_odd), dtype=dtype,
                        device=device, generator=gen)
        try:
            for _ in range(nv_iters):
                # 相对单线程版收敛容差：C++ matvec 噪声底 ~2e-6 (float32)，
                # 5e-5 绝对容差导致残差卡噪声层不收敛（max_iter 空转）。
                v = v - solver.bistabcg(b=mv(v), matvec=mv, tol=5e-5, verbose=False)
            nrm = torch.linalg.norm(v)
            if not torch.isfinite(v).all() or float(nrm) == 0.0:
                continue

```

```

        v = v / nrm
        if torch.isfinite(v).all():
            return v
    except RuntimeError:
        # BiCGStab breakdown on random rhs: resample and retry
        continue
# 最终回退：归一化随机向量（避免 zeros 导致 norm 除零）
if v is None or not torch.isfinite(v).all() or float(torch.linalg.norm(v)) == 0.0:
    v = torch.randn([e] + list(lat_fine_odd), dtype=dtype,
                    device=device, generator=gen)
    v = v / torch.linalg.norm(v)
return v

def worker(tid):
    # 显式绑定设备上上下文（与 MultiGpuMultigrid 约定一致）：worker 线程
    # 首次 CUDA 操作须 set_device，否则 torch per-thread 默认流/CUDA 状态
    # 与 C++ 后端调用环境不一致，导致多线程下 matvec 结果异常。
    torch.cuda.set_device(device.index if (device.type == 'cuda' and device.index is not None) else
                          0)
    op = matvec_ops[tid % len(matvec_ops)]
    c0 = tid * chunk
    c1 = min(dof, c0 + chunk)
    for i in range(c0, c1):
        null[i] = _gen(op, i, tid)

if nthreads <= 1:
    # 单线程直接执行（避免嵌套线程池触发 torch lazy-wrapper 冲突）
    worker(0)
else:
    with ThreadPoolExecutor(max_workers=nthreads) as ex:
        list(ex.map(worker, range(nthreads)))
if verbose:
    import time

```

5 参考源清单

文件: 行号	类型	内容
pyqcu/cuda/qcu/qcu.pyx:1-16	仓库	Cython 桥 nogil 真并行
pyqcu/cuda/qcu/qcu_api.pxd	仓库	pxd 别名（nogil 声明）
pyqcu/cuda/_schur_op.py:47-95	仓库	CudaSchurOp + 深层竞争修复
pyqcu/cuda/_multi_gpu.py:45-390	仓库	多线程多卡驱动 + 独立模式
pyqcu/tools/_multigrid.py:281-410	仓库	多线程 null/stencil 构建
pyqcu/tools/_io.py:165-200	仓库	h5py 多线程安全读写
pyqcu/solver/_multigrid.py:357-395	仓库	同步 + breakdown 自动重启
pyqcu/testing/___init__.py:535-630	仓库	多线程验证测试
pyqcu/cuda/AGENTS.md	仓库	已知问题（2L T 不粗化越界）

6 大格子与多配置验证 (v3 新增)

- **16^4 2L**: mt4 构建 1358s (E=48 首次)、C++ MG 求解 5.7s、残差 1.5×10^{-7} PASS; **缓存命中 2.3s** (590x 加速)。
- **$12^3 \times 16$ 2L (非 2 幂)**: mt4 构建 398s、求解 5.6s、残差 1.5×10^{-7} PASS。
- **confest.clover.multigrid.py 协议迁移**: 旧协议 (base=10 3 槽、无 hop_diag、E=12、T 参数错误) 与 C++ 新协议 (base=30 4 槽 33-tensor) 不符是 2L 越界/nan 的根因 (非内核 bug); 迁移后 $8 \times 8 \times 8 \times 16$ 2L/2L_r3、 $12^3 \times 16$ 、 16^4 全 PASS。
- **Python multigrid 多线程实例**: 2 线程独立实例并行求解 $8 \times 8 \times 8 \times 8$ 1L 残差 $8.9 \times 10^{-6} / 9.5 \times 10^{-6}$ PASS; MultiGpuMultigrid.solve 内置 torch lazy 预热 (防 worker 并发 torch.linalg.inv 初始化竞争)。
- **参数扫描**: 8 配置 (L2/3、r5-20、ct1e2-1e5) 全部收敛 (1.5×10^{-7})。

7 结论与局限

结论

最终结论:

1. 多线程多卡 MultiGrid 路径完整可用: Cython 桥真并行、算子线程安全、驱动双模式、粗网格构建加速 2.8x; 正确性以多线程一致性与 stencil 自洽性双重判据实测达标。
2. 本轮修复了 5 类真实缺陷 (params 维度/非连续输出/流同步/深层内存竞争/混合路径同步 + 数值鲁棒性), 全部有可复现证据与回归。
3. 持久化统一 h5py 且多线程安全; 散落库代码 (Schur 算子、stencil 构建) 已合并。

局限:

- 本机单卡 (RTX 4060 8GB): 多卡物理部署需多 GPU 节点实测; 单卡多线程 GPU 瓶颈 (构建 2.8x 为线程并行 + C++ matvec 的双重收益, 求解阶段单卡无加速)。
- C++ 后端对”T 方向不粗化”2 层配置越界 (lattice_clover_multigrid.h:1530) 为已知问题, 修复需深入内核粗层尺寸假设, 暂缓。
- Python multigrid 求解器 (非 C++ 路径) 性能瓶颈在 Python 层与同步开销, C++ 后端为生产路径。